Program 4: Implement Merge Sort using Array

Objective:

To implement Merge Sort using an array and arrange the elements in ascending order.

Theory:

Merge Sort is a divide-and-conquer sorting technique. The array is divided into two halves until single-element subarrays are obtained. The sorted halves are then merged to produce the final sorted array.

Algorithm:

Step 1: Start.
Step 2: Enter the number of elements and the array elements.
Step 3: Divide the array into two halves.
Step 4: Recursively sort the left half.
Step 5: Recursively sort the right half.
Step 6: Merge the two sorted halves by comparing their elements.
Step 7: Repeat the process until the complete array is sorted.
Step 8: Display the sorted array.
Step 9: Stop.

Complexity:

Time Complexity: O(n log n)
Space Complexity: O(n)

Program:

```
#include <stdio.h>

void merge(int arr[], int low, int mid, int high) {
    int temp[100], i = low, j = mid + 1, k = 0;

    while (i <= mid && j <= high) {
        if (arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while (i <= mid)
        temp[k++] = arr[i++];

    while (j <= high)
        temp[k++] = arr[j++];

    for (i = low, k = 0; i <= high; i++, k++)
        arr[i] = temp[k];
}

void mergeSort(int arr[], int low, int high) {
    if (low >= high)
        return;
```

```
    int mid = (low + high) / 2;

    mergeSort(arr, low, mid);
    mergeSort(arr, mid + 1, high);
    merge(arr, low, mid, high);
}

int main() {
    int n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter array elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    mergeSort(arr, 0, n - 1);

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

Sample Output



Program 5: Implement Heap Sort using Array

Objective:

To implement Heap Sort using an array and arrange the elements in ascending order.

Theory:

Heap Sort is a comparison-based sorting technique that uses a binary heap. For ascending order, a max heap is constructed and the largest element is repeatedly placed at the end of the array.

Algorithm

Step 1: Start.
Step 2: Enter the number of elements and the array elements.
Step 3: Build a max heap from the given array.
Step 4: Compare each parent with its children and maintain the max-heap property.
Step 5: Swap the root element with the last element of the unsorted part.
Step 6: Reduce the heap size and heapify the root again.
Step 7: Repeat until all elements are sorted.
Step 8: Display the sorted array.
Step 9: Stop.

Complexity:

Time Complexity: O(n log n)
Space Complexity: O(1)

Program:

```
#include <stdio.h>

void heapify(int arr[], int n, int i) {
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;
   int temp;

   if (left < n && arr[left] > arr[largest])
      largest = left;

   if (right < n && arr[right] > arr[largest])
      largest = right;

   if (largest != i) {
      temp = arr[i];
      arr[i] = arr[largest];
      arr[largest] = temp;

      heapify(arr, n, largest);
   }
}

void heapSort(int arr[], int n) {
   int i, temp;

   for (i = n / 2 - 1; i >= 0; i--)
      heapify(arr, n, i);
```

```
    for (i = n - 1; i > 0; i--) {
        temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}

int main() {
    int n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter array elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    heapSort(arr, n);

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

Sample Output:



Program 6: Implement Quick Sort using Array

Objective

To implement Quick Sort using an array and arrange the elements in ascending order.

Theory

Quick Sort is a divide-and-conquer sorting technique. A pivot element is selected and the array is partitioned so that smaller elements are placed before the pivot and larger elements after it. The same process is recursively applied to both parts.

Algorithm

Step 1: Start.

Step 2: Enter the number of elements and the array elements.

Step 3: Select the last element as the pivot.

Step 4: Partition the array around the pivot.

Step 5: Place the pivot at its correct position.

Step 6: Recursively apply Quick Sort to the left and right parts.

Step 7: Repeat until each part contains at most one element.

Step 8: Display the sorted array.

Step 9: Stop.

Complexity

Time Complexity: Average: O(n log n), Worst: $O(n^2)$
Space Complexity: O(log n) average recursion stack; O(n) worst case

Program

```
#include <stdio.h>

int partition(int arr[], int low, int high) {

    int pivot = arr[high];

    int i = low - 1;

    int j, temp;

    for (j = low; j < high; j++) {

        if (arr[j] <= pivot) {

            i++;

            temp = arr[i];

            arr[i] = arr[j];

            arr[j] = temp;

        }

    }
```

```
    temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter array elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    quickSort(arr, 0, n - 1);
    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

Sample Output



Program 7: Implement Insertion Sort using Array

Objective

To implement Insertion Sort using an array and arrange the elements in ascending order.

Theory

Insertion Sort is a simple sorting technique in which the array is divided into a sorted and an unsorted part. Each element from the unsorted part is inserted into its correct position in the sorted part.

Algorithm

Step 1: Start.

Step 2: Enter the number of elements and the array elements.

Step 3: Consider the first element as the sorted part.

Step 4: Select the next element as the key.

Step 5: Compare the key with the elements in the sorted part.

Step 6: Shift all elements greater than the key one position to the right.

Step 7: Insert the key at its correct position.

Step 8: Repeat until all elements are inserted.

Step 9: Display the sorted array.

Step 10: Stop.

Complexity

Time Complexity: $O(n^2)$
Space Complexity: $O(1)$

Program

```
#include <stdio.h>


int main() {
    int n, i, j, key;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter array elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
```

```
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

Sample Output

